

Matrix Data Engineering

Updated evaluation guide — current project status

Contents

1. [Project overview](#)
2. [Exercise 0 — construct.py](#)
3. [Exercise 1 — loading.py](#)
4. [Exercise 2 — oracle.py](#)
5. [Supporting files](#)
6. [Important concepts](#)
7. [Testing checklist](#)
8. [Evaluation questions and answers](#)
9. [Final review notes](#)

1. Project overview

The project teaches three related pieces of real-world Python and data-engineering practice:

1. **Isolation:** use a virtual environment so packages for one project do not interfere with other projects.
2. **Dependencies:** describe and install external libraries with pip and Poetry.
3. **Configuration:** keep environment-specific settings and secrets outside the source code.

Exercise	Files	Lesson
0	<code>construct.py</code>	Detect and inspect virtual environments.
1	<code>loading.py</code> , <code>requirements.txt</code> , <code>pyproject.toml</code>	Check dependencies, simulate data, calculate profiles, and generate a chart.
2	<code>oracle.py</code> , <code>requirements.txt</code> , <code>.env</code> , <code>.env.example</code> , <code>.gitignore</code>	Load, validate, and protect configuration.

General subject rules

- Python 3.10 or later.
- flake8-compatible style.
- Comprehensive type annotations checked with mypy.
- Exception handling that protects the data workflow from corruption.
- Testing with and without virtual environments, dependencies, and configuration.

2. Exercise 0 — construct.py

Purpose

`construct.py` determines whether the current Python interpreter is inside a virtual environment. It then prints the active interpreter, environment name, package location, or instructions for creating a virtual environment.

Imports

Import	Role
<code>sys</code>	Provides information about the running interpreter, including <code>prefix</code> , <code>base_prefix</code> , and <code>executable</code> .
<code>os</code>	Reads <code>VIRTUAL_ENV</code> and extracts folder names with <code>os.path.basename()</code> .
<code>site</code>	Finds Python package directories with <code>getsitepackages()</code> and <code>getusersitepackages()</code> .

Program flow

1. The `if __name__ == "__main__"` block calls `main()` when the file is run directly.
2. `in_virtual_environment()` compares `sys.prefix` with `sys.base_prefix`.
3. If the values differ, the current interpreter belongs to a virtual environment.
4. `get_environment_name()` checks `VIRTUAL_ENV` and extracts its final folder name. If unavailable, it uses the final part of `sys.prefix`.
5. `get_package_path()` returns the first system or virtual environment package directory, or the user package directory as a fallback.
6. Inside a venv, the program prints “Welcome to the construct”, the interpreter path, venv name, venv path, and package installation path.
7. Outside a venv, it prints “You’re still plugged in”, warns about global installation, and shows commands to create and activate a venv.

```
python3 -m venv matrix_env
source matrix_env/bin/activate      # Unix/macOS
matrix_env\Scripts\activate        # Windows
deactivate
```

Current output choice

The latest version places the heading inside each branch:

```
MATRIX STATUS: Welcome to the construct

# or

MATRIX STATUS: You're still plugged in
```

This is clearer because the status and the branch result are printed together.

3. Exercise 1 — loading.py

Purpose

The current Exercise 1 simulates 1,000 people connected to the Matrix. It calculates an awakening score from two percentage indicators, assigns each subject a profile, counts the profiles, and saves a bar chart to `matrix_analysis.png`.

Current configuration

Item	Current value	Meaning
Required packages	NumPy, pandas, matplotlib	Simulation, data table and analysis, visualization.
Matrix signal	Normal distribution, <code>loc=5</code> , <code>scale=90</code> , clipped to 0–100	Widely varying percentage-like signal.
Resistance rate	Uniform distribution from 0 to 100	Every range of resistance is equally likely.
Awakening score	60% signal + 40% resistance	Combined measure used for classification.
Profile bins	0–30, 30–60, 60–75, 75–100	Four awakening categories.

Imports and constants

Item	Use
<code>sys</code>	Calls <code>sys.exit(1)</code> when required dependencies are missing.
<code>metadata</code>	Uses <code>metadata.version()</code> to inspect installed package versions.
<code>Final</code>	Marks configuration values such as dependencies, count, bins, labels, and colors as constants.
<code>DEPENDENCIES</code>	Maps package names to their purposes for readable status output.
<code>SUBJECT_COUNT</code>	Centralizes the value 1,000.
<code>PROFILE_BINS</code>	Stores the numerical profile boundaries.
<code>PROFILE_LABELS</code>	Stores <code>Dormant</code> , <code>Questioning</code> , <code>Aware</code> , and <code>Awakened</code> .
<code>PROFILE_COLORS</code>	Stores the four custom bar colors.

Program flow

- `main()` calls `check_dependencies()` .
- The dependency checker asks `importlib.metadata` for the installed version of each required package.
- If a package is absent, it is added to a list and installation instructions are shown later.
- `show_dependency_management_difference()` prints the pip versus Poetry explanation.
- If the missing-package list is not empty, the program prints help and exits with status 1.
- Otherwise, `run_analysis()` imports matplotlib, NumPy, and pandas locally inside a `try/except ImportError` .
- NumPy generates 1,000 matrix-signal values. The normal distribution uses `loc=5` as its center and `scale=90` as its very wide spread.
- `np.clip(..., 0, 100)` forces the signal to remain a valid percentage.
- NumPy generates 1,000 resistance values uniformly between 0 and 100.
- pandas creates a DataFrame with both columns.
- The DataFrame receives an `awakening_score` column using the weighted formula.
- `pnd.cut()` assigns each score to one profile.
- `value_counts(sort=False)` counts the subjects while preserving the profile order.
- The counts are printed and plotted.
- The chart is formatted, saved as `matrix_analysis.png` , and closed.

Awakening-score calculation

```
data_frame["awakening_score"] = (  
    data_frame["matrix_signal"] * 0.6  
    + data_frame["resistance_rate"] * 0.4  
)
```

This is a weighted average:

$$0.6 + 0.4 = 1.0$$

Matrix signal has the greater influence because it is the primary simulated indicator of awareness. Resistance rate contributes a smaller but meaningful amount.

Example:

```
matrix_signal = 50  
resistance_rate = 80  
  
awakening_score = 50 × 0.6 + 80 × 0.4  
awakening_score = 62
```

Profile categories

Score interval	Profile	Interpretation
0–30	Dormant	Little simulated evidence of questioning.
More than 30–60	Questioning	Some signs of awareness or resistance.
More than 60–75	Aware	Strong signs of seeing the Matrix differently.
More than 75–100	Awakened	Highest simulated likelihood of rejecting the Matrix.

The slightly extended boundaries `-0.1` and `100.1` ensure exact 0 and 100 values are included when pandas creates its intervals.

4. Exercise 2 — oracle.py

Purpose

`oracle.py` demonstrates configuration management. It loads five settings from the environment and local `.env` file, checks that they are present and valid, shows local versus production behavior, and reports configuration status without printing private values.

Current imports

Import	Use
<code>import os</code>	Reads variables with <code>os.getenv()</code> and checks <code>.env</code> with <code>os.path.exists()</code> .
<code>import sys</code>	Stops execution with status 1 when <code>python-dotenv</code> or required configuration is unavailable.
<code>from dotenv import load_dotenv</code>	Comes from <code>python-dotenv</code> and loads local <code>.env</code> key/value pairs.

Current configuration names

Variable	Purpose
<code>MATRIX_MODE</code>	Must be <code>development</code> or <code>production</code> .
<code>DATABASE_URL</code>	Database connection configuration.
<code>API_KEY</code>	Credential for an external service.
<code>LOG_LEVEL</code>	Logging verbosity setting.
<code>ZION_ENDPOINT</code>	External network or service endpoint.

Program flow

1. The program tries to import `load_dotenv`. If `python-dotenv` is not installed, it prints the pip requirements command and exits.
2. `main()` prints the Oracle status heading.
3. `load_configuration()` calls `load_dotenv()` and then reads all five settings with `os.getenv()`.
4. The values are returned in one dictionary. Missing values are represented by `None`.

5. `validate_configuration()` checks every required name and records missing or empty values in `config_issues`.
6. It also checks that `MATRIX_MODE` is exactly `development` or `production`.
7. If there are issues, the program prints each one, prints an incomplete configuration status, runs the security check, and exits with code 1.
8. If configuration is valid, `print_configuration()` prints a compact status report.
9. It reports a local database for development and a production database for production.
10. It reports API access as authenticated when `API_KEY` exists, but does not print the key.
11. It reports the Zion network as online when `ZION_ENDPOINT` exists.
12. It reports only `Database URL: Configured` or `Database URL: Missing`, avoiding disclosure of the actual connection string.
13. `print_security_check()` reports whether `.env` exists and reminds the user that production environment variables can override development values.

What this program does not do

- It does not connect to a real database.
- It does not contact the API.
- It does not contact the Zion endpoint.
- It does not parse a custom configuration format.
- It does not print or mask the API key; it only checks whether one exists.

Those behaviors are intentional. The exercise is about loading and managing configuration securely, not implementing external services.

5. Important concepts

Virtual environment

A venv is an isolated Python environment with its own interpreter path and package directory. It lets each project install the versions it needs without changing global Python.

Dependency

A dependency is an external library required by a program. Dependencies are recorded in files so another person can recreate the environment.

pip versus Poetry

pip installs packages listed in `requirements.txt` into the current environment. Poetry reads `pyproject.toml`, resolves compatible versions, and manages a project environment. Pip is simpler; Poetry is more project-oriented.

Environment variable

An environment variable is a value provided outside the source code, such as `API_KEY=...`. The same program can receive different values on different machines without changing the Python file.

.env loading and precedence

`load_dotenv()` loads local development values. Operating-system environment variables can override those values, which is useful for production deployment. The Python program accesses both through `os.getenv()`.

Secret versus configuration

A database URL may be ordinary configuration, but it can also contain a password. An API key is a secret credential. Neither should be hardcoded or exposed in Git. The current Oracle design reports whether values exist without printing the API key or database URL.

Validation

Loading means retrieving values. Validation means checking whether those values exist and make sense. Oracle validates presence of all five settings and restricts `MATRIX_MODE` to two accepted values.

Type annotations

The subject asks for comprehensive annotations checked with mypy. Your functions use annotations such as `dict[str, str | None]` and `list[str]`. `Final` is used in Exercises 0 and 1 as a helpful constant marker, but it is not itself the main requirement.

6. Testing checklist

Test	Expected result
Run <code>construct.py</code> outside a venv	Global-environment warning and venv creation instructions.
Run <code>construct.py</code> inside a venv	Venv name, interpreter path, environment path, and package path.
Run <code>loading.py</code> without dependencies	Missing package list and pip/Poetry installation instructions.
Run <code>loading.py</code> with pip dependencies	Versions, profile counts, and <code>matrix_analysis.png</code> .
Run <code>loading.py</code> through Poetry	<code>poetry install</code> and <code>poetry run python loading.py</code> work.
Run Oracle without <code>.env</code> or environment variables	Missing-variable warnings and exit code 1.
Run Oracle with complete <code>.env</code>	Configuration-loaded report and security check.
Set <code>MATRIX_MODE=production</code> externally	Production mode appears instead of development mode.
Use invalid mode, such as <code>testing</code>	Invalid-mode warning and safe exit.
Run Oracle without python-dotenv	Dependency warning and installation command.

7. Evaluation questions and answers

Why are virtual environments useful?

They isolate project dependencies, avoid version conflicts, protect global Python, and make it easier to reproduce a project on another machine.

How does `construct.py` detect a venv?

It compares `sys.prefix`, the active environment location, with `sys.base_prefix`, the base Python location. Different values indicate a virtual environment.

What does `pip` do here?

`pip` installs packages listed in `requirements.txt` into the currently active Python environment.

What does Poetry do here?

Poetry reads `pyproject.toml`, resolves dependencies, and manages the project environment. The program prints the command needed to install and run through Poetry.

Why does `loading.py` import the external libraries inside `run_analysis()`?

The program first checks whether the packages are available. Delaying the imports lets it print friendly installation instructions instead of failing immediately at the top of the file.

What is the awakening score?

It is a weighted average: 60% Matrix signal plus 40% resistance rate. It combines two percentage indicators into one score used for profile classification.

Why clip Matrix signal to 0–100?

The normal distribution can generate values below 0 or above 100. Since the signal is treated as a percentage, `np.clip()` forces every result into the valid range.

What do `loc` and `scale` mean?

`loc` is the center or average of a normal distribution. `scale` is the standard deviation, which controls spread. A scale of 90 creates a very wide distribution, so clipping is especially important.

How are profiles assigned?

pandas `cut()` places each awakening score into one interval: 0–30 Dormant, over 30–60 Questioning, over 60–75 Aware, and over 75–100 Awakened.

What does oracle.py do?

It loads five configuration values from environment variables and a local `.env` file, validates them, shows development versus production behavior, and reports status without exposing private values.

What happens if a configuration variable is missing?

The variable name is added to `config_issues`. The program prints every issue, reports that configuration is incomplete, runs the security check, and exits with code 1.

Why use python-dotenv?

It loads simple `KEY=value` entries from `.env` into the process environment. The assignment specifically requires using this library rather than writing a custom parser.

Why should .env not be committed?

It may contain API keys, passwords, or private connection strings. Committing it could expose secrets. `.env.example` is committed instead, with safe placeholders.

Why is .gitignore committed?

The `.gitignore` file shares the repository's protection rules with other developers. The real `.env` is ignored; `.env.example` and `.gitignore` themselves are safe project files and should be committed.

Does Oracle actually connect to the database or Zion endpoint?

No. It only validates and reports configuration. The exercise is about configuration management, not implementing real external connections.

8. Final review notes

Verify before submission:

- Exercise 2 `requirements.txt` contains `python-dotenv`.
- The real `.env` is not shown by `git status`.
- `.env.example` contains all five required names and only safe placeholder values.
- `.gitignore` contains `.env`.
- Run all three programs from the directories where their relative files are expected.
- Run `flake8` and `mypy` according to the project's setup.
- Demonstrate both valid and invalid or missing configuration cases for Oracle.
- Remember that `DATABASE_URL` can contain credentials in real applications, so reporting only “Configured” is safer than printing its value.

You should be able to explain why each tool exists, what happens when something is missing, and how the project keeps environments, dependencies, and configuration separate.